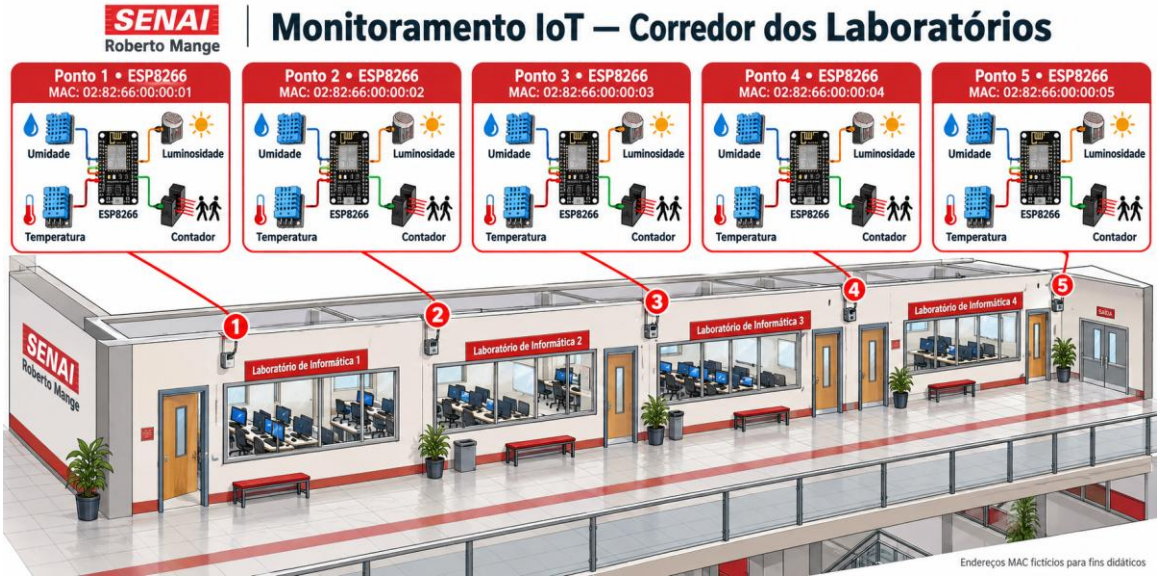
ATIVIDADE	PROJETO INTEGRADOR (PWBE)

CONTEXTO e DESCRIÇÃO

A escola TecnoVille deseja monitorar as condições ambientais e a circulação de pessoas nos corredores próximos aos laboratórios de informática. Para isso, serão instalados cinco pontos de monitoramento, cada um equipado inicialmente com um ESP8266 e sensores de temperatura, umidade, luminosidade e contagem de passagens.

Esses sensores serão instalados em locais como os corredores.



A equipe responsável precisa identificar onde cada equipamento está instalado e conhecer seu modelo, endereço MAC e coordenadas geográficas. Também deverá ser possível registrar observações, consultar a data de cadastro e indicar se o equipamento está ativo ou inativo. O sistema deverá permitir cadastrar novos locais e modelos de equipamentos conforme a expansão do projeto.

Cada microcontrolador poderá estar conectado a vários sensores. Será necessário identificar a função de cada sensor, seu modelo, sua unidade de medida, seu estado de ativação e o equipamento ao qual está ligado. As medições deverão utilizar graus Celsius para temperatura, porcentagem para umidade, lux para luminosidade e unidades para contagem de passagens.

Os dados recebidos deverão permanecer armazenados para consultas posteriores. Para cada leitura, o sistema deverá permitir identificar o sensor responsável, o valor medido e a data e hora da medição. Neste projeto, a contagem representará a quantidade de passagens detectadas no intervalo de coleta.

A equipe escolar deverá conseguir consultar as leituras de um sensor específico, selecionar um período e localizar os dados dos equipamentos instalados em determinado ponto. Sensores inativos não poderão registrar novas leituras. Desativar um equipamento ou sensor deverá preservar os dados já coletados.

O acesso ao sistema será realizado por usuários autenticados. Cada usuário terá informações de identificação, credenciais de acesso e uma foto. Serão considerados três perfis: Superusuário, Administrador e Usuário. Superusuários e administradores poderão cadastrar, consultar, editar e excluir registros, respeitando a integridade dos dados. Usuários comuns poderão apenas consultar as informações.

Com base nessas necessidades, cada aluno deverá elaborar sua própria modelagem de banco de dados, definindo as entidades, os campos, os tipos de dados, as obrigatoriedades, as restrições de unicidade e os relacionamentos necessários. Os nomes das tabelas e dos campos serão escolhidos pelo aluno.

O Diagrama Entidade-Relacionamento deverá ser elaborado pelo aluno e entregue em formato JPG no repositório. A implementação em Django e Django REST Framework deverá corresponder ao diagrama produzido e atender às regras descritas neste enunciado.

Autenticação e permissões

Utilize um modelo de usuário baseado em `AbstractUser` e implemente autenticação com JSON Web Tokens (JWT).

O **token de acesso** deverá ter validade de **25 minutos**. O **token de atualização (*refresh token*)** deverá ter validade máxima de **12 horas**. Após esse período, o usuário precisará realizar um novo login.

Para a avaliação em ambiente de desenvolvimento, crie um **superusuário** com as credenciais:

- **Username:** `lin`
- **Password:** `123`

Para os outros usuários colocar as credenciais:

- **Username:** `senai`
- **Password:** `123456`

Configure as permissões da API de acordo com o tipo de usuário:

- **Superusuário e Administrador:** pode cadastrar, consultar, editar e excluir os dados.
- **Usuário:** pode apenas consultar os dados.

As permissões devem ser verificadas nos endpoints da API, inclusive quando uma requisição for feita diretamente pelo Postman ou Insomnia.

Funcionalidades obrigatórias

Desenvolva uma API com Django e Django REST Framework que permita:

- Cadastrar, listar, consultar, editar e excluir locais, modelos de equipamentos, microcontroladores e sensores, conforme a modelagem elaborada pelo aluno.
- Cadastrar e gerenciar usuários, incluindo foto e perfil de acesso.
- Ativar e desativar microcontroladores e sensores, preservando as leituras já armazenadas.
- Receber e armazenar leituras de temperatura, umidade, luminosidade e contagem de passagens, identificando o sensor responsável, o valor e a data e hora da medição.
- Impedir o registro de novas leituras quando o sensor ou o microcontrolador correspondente estiver inativo.
- Consultar o histórico de leituras de um sensor específico.
- Filtrar as leituras por sensor, período e local de instalação.
- Filtrar microcontroladores e sensores por status ativo ou inativo.

- Impedir que a exclusão de registros relacionados apague os históricos de leitura.
- Realizar login e renovar o acesso por meio de tokens JWT, conforme os prazos definidos na seção de autenticação.
- Aplicar as permissões de cada perfil em todos os endpoints: Superusuário e Administrador poderão gerenciar os dados; Usuário poderá apenas consultar. Usuários comuns não poderão alterar o próprio perfil para obter privilégios.

Importação, simulação e testes

Com base na modelagem elaborada, o aluno deverá utilizar uma ferramenta de IA para gerar arquivos CSV destinados à população do banco de dados. Os arquivos deverão conter **pelo menos cinco microcontroladores, quatro sensores por microcontrolador e 100 leituras distribuídas entre os quatro tipos**, deixando os demais cadastros em quantidade suficiente para demonstrar os relacionamentos.

Os dados deverão ser coerentes com as regras do sistema, respeitando os campos, os valores permitidos, as restrições de unicidade e os relacionamentos definidos pelo aluno.

O aluno deverá desenvolver um script para importar esses arquivos. A importação deverá ser demonstrada com os registros gravados no banco e disponíveis para consulta pela API.

Também deverá utilizar uma ferramenta de IA para gerar arquivos `.sql` com dados adicionais para popular o banco MySQL. Os comandos deverão ser compatíveis com as tabelas criadas pelas migrações do Django e respeitar a ordem de inserção exigida pelas chaves estrangeiras.

Cabe ao aluno revisar os arquivos gerados, corrigir eventuais erros e verificar sua compatibilidade com a aplicação. O uso de IA não dispensa a compreensão do conteúdo produzido.

Na raiz do repositório, deverá ser criada uma pasta chamada **data**, contendo os arquivos CSV, os arquivos SQL e os prompts utilizados para gerá-los. Os nomes dos arquivos deverão identificar claramente as entidades correspondentes, conforme a modelagem adotada.

As rotinas de importação e população deverão ser organizadas no arquivo **popula.py**, no mesmo diretório de `manage.py`, e executadas pelo console, com acesso ao ambiente do Django. Essas rotinas não deverão ser colocadas em `views.py`.

No cadastro de usuários, as senhas deverão ser armazenadas como hashes válidos do Django. Não será aceito o armazenamento de senhas em texto puro.

Utilizando o Insomnia, o aluno deverá testar e demonstrar:

- Cadastro, consulta, edição e exclusão dos registros.
- Recebimento e consulta das leituras dos quatro tipos de sensores.
- Filtros por sensor, período, localização e status, conforme os requisitos.
- Login, renovação do token e permissões de acesso.
- Bloqueio de alterações realizadas por usuários comuns.
- Bloqueio de novas leituras de sensores ou microcontroladores inativos.
- Preservação dos históricos diante de tentativas de exclusão de registros relacionados.

Os dados utilizados nos testes poderão ser simulados, sem necessidade de conexão com dispositivos físicos.

Funcionalidades extras

Implemente **pelo menos cinco** dos recursos abaixo. Eles deverão ser demonstrados na avaliação e não substituem as funcionalidades obrigatórias:

- **Ordenação nas consultas:** permitir ordenar os resultados por campos pertinentes, como nome, data da leitura ou valor medido.
- **Resumo estatístico por sensor:** disponibilizar um endpoint que apresente quantidade de leituras, valor mínimo, máximo e média em um período.
- **Alertas de medições:** permitir definir limites por sensor e registrar alertas quando uma leitura estiver fora desses limites.
- **Detecção de sensores sem comunicação:** identificar sensores ativos que não enviaram leituras dentro de um intervalo definido.
- **Prevenção de leituras duplicadas:** estabelecer e implementar uma regra para impedir que a mesma leitura seja registrada mais de uma vez.
- **Testes automatizados:** implementar testes que verifiquem regras de negócio, autenticação e permissões da API.
- **Documentação automática com Swagger:** documentar os endpoints, os parâmetros e os dados enviados e recebidos.
- **Geração de leituras simuladas:** criar um comando personalizado do Django para gerar e armazenar medições coerentes com os tipos de sensores.
- **Registro de manutenção:** permitir registrar e consultar intervenções realizadas nos sensores, incluindo data, descrição e responsável.
- **Auditoria de operações:** registrar o usuário, a ação realizada, o registro afetado e a data e hora das alterações.
- **Consulta rápida de leituras recentes:** disponibilizar uma consulta específica que retorne a última leitura de cada sensor e uma opção para consultar automaticamente as últimas 24 horas, sem informar manualmente as datas.

Cada recurso escolhido deverá ser identificado no README do repositório, com uma breve descrição e instruções para sua demonstração.

Entrega e avaliação

O trabalho é **individual**. O projeto deverá estar em um **repositório privado**, com acesso concedido ao professor pelo e-mail **lindomarbatistao@gmail.com**.

A data de entrega fica para o dia **30/11/2026**.

Na data de entrega, o professor avaliará diretamente o estado do repositório naquele momento. Alterações posteriores não serão consideradas.

Durante a execução, as dúvidas encaminhadas ao professor deverão tratar exclusivamente da **interpretação do enunciado**. Não haverá orientação técnica detalhada sobre a implementação.

Em caso de suspeita de plágio ou cópia parcial ou total, os trabalhos envolvidos serão analisados; a aplicação de **nota zero** dependerá da apuração da situação.

Observações: a correção irá acontecer no Insomnia, portanto os dados do CRUD serão manipulados via JSON com exceção da foto do usuário, mas a população do banco deverá ser via console

Critérios de avaliação

Critérios críticos — 50 pontos

Correspondem ao funcionamento básico da aplicação, demonstrado localmente.

Nº	Critério	Pontos
1.	Projeto Django configurado com Django REST Framework, aplicação, URLs e dependências necessárias	4
2.	Elaboração e entrega do DER em formato JPG no repositório	7
3.	Chaves estrangeiras e relacionamentos do DER funcionando	2
4.	Banco de dados em MySQL	3
5.	Tratamento de dados inválidos na API, com mensagens claras e códigos HTTP adequados para campos obrigatórios ausentes, valores incorretos e registros inexistentes	4
6.	Migrações aplicadas e banco local funcionando	4
7.	`TextChoices` implementados para dados padrão	5
8.	Serializers funcionando	2
9.	CRUD dos cadastros e registro e consulta das leituras, respeitando os relacionamentos	6
10.	Endpoints para cadastrar e listar dados de temperatura, umidade, contador e luminosidade	5
11.	Login por JWT funcionando, com emissão de token de acesso	4
12.	Preservação dos históricos e proteção dos registros relacionados.	4
	Total	50

Critérios desejáveis — 50 pontos

Avaliam regras de negócio, operação nos dois serviços de hospedagem e domínio da solução.

Nº	Critério	Pontos
13.	Filtros por sensor, período, status e localização	6
14.	Validações de regras de negócio, incluindo compatibilidade entre sensor e leitura e bloqueio de leituras de **microcontroladores inativos**, além dos sensores.	4
15.	Token de acesso com validade de 25 minutos, refresh token com validade máxima de 12 horas e renovação funcionando	5
16.	Permissões por tipo de usuário: superusuário e administrador realiza CRUD; usuário apenas consulta	5
17.	Cadastro correto da foto e do tipo na tabela de usuários	3
18.	Script que importa corretamente os dados dos arquivos CSV gerados e revisados pelo aluno	4
19.	Arquivos `.sql` gerado com auxílio de IA, revisado e executado no MySQL, com prompt e arquivos organizados na pasta `DATA`	3
20.	Arquivos .csv gerado com auxílio de IA, revisado e executado no MySQL, com prompt e arquivos organizados na pasta `DATA`	3
21.	API publicada e funcionando no **PythonAnywhere ou AlwaysData**, com dados importados e endpoints de cadastro, consulta, autenticação e permissões testados no Insomnia.	5
22.	Pelo menos 5 funcionalidades extras implementadas e demonstradas	3
23.	README com instruções para instalação, configuração, população do banco e execução da API, incluindo a identificação das funcionalidades extras implementadas	5
24.	Explicação oral de algum trecho do código escolhido pelo professor	4
	Total	50